

Documentación EyEDooR

José Daniel Salcedo Gutiérrez
Antonio Alejandro Betancourt Torregroza
Yheiner Daniel Gomez Mican

3 de diciembre de 2025

Índice

1. Resumen del Proyecto	3
1.1. Flujo General del Sistema	3
2. Backend (Python/Flask)	3
2.1. Archivo Principal: app.py	3
2.2. Paquetes y Librerías del Backend	4
2.2.1. Flask (<code>flask</code>)	4
2.2.2. Werkzeug (<code>werkzeug.security</code>)	4
2.2.3. Firebase Admin SDK (<code>firebase-admin</code>)	5
2.2.4. Requests (<code>requests</code>)	5
2.2.5. OS (<code>os</code>)	6
2.2.6. RE (<code>re</code>)	6
2.2.7. Datetime (<code>datetime</code>)	6
2.3. Rutas Principales del Backend	6
2.3.1. Autenticación	6
2.3.2. Panel Principal	6
2.3.3. API de Cámara	6
2.3.4. Debug	7
2.4. Sistema de Validación de Códigos de Producto	7
3. Frontend (HTML/CSS/JavaScript)	7
3.1. Archivos de Plantillas (carpeta <code>templates/</code>)	7
3.1.1. login.html	7
3.1.2. registro.html	7
3.1.3. recuperar.html	8
3.1.4. bienvenido.html (Página Principal)	8
3.2. Estilo CSS	11
4. Hardware (Arduino/ESP32)	11
4.1. Hardware 1: ESP32 con Botón de Timbre	11
4.1.1. Archivo: <code>arduino/boton/boton.ino</code>	11
4.1.2. Librerías de Arduino del Botón	11

4.1.3.	Configuración del Hardware	12
4.1.4.	Sincronización NTP	12
4.1.5.	Loop Principal del Botón	13
4.2.	Hardware 2: ESP32-CAM	14
4.2.1.	Archivo: <code>arduino/camara/camara.ino</code>	14
4.2.2.	Librerías de Arduino de la Cámara	14
4.2.3.	Configuración de la Cámara	15
4.2.4.	Arquitectura de Dual-Server	15
4.2.5.	Inicialización de Servidores	16
4.2.6.	Endpoints de la ESP32-CAM	17
5.	Flujo Completo de Captura de Foto	17
5.1.	Escenario 1: Timbre Físico Presionado	17
5.2.	Escenario 2: Botón Web Presionado	17
6.	Estructura de Datos en Firebase	18
6.1.	Nodo <code>/users</code>	18
6.2.	Nodo <code>/timbres</code>	18
6.3.	Nodo <code>/fotos.eyedoor</code>	18
7.	Consideraciones de Seguridad	18
7.1.	Buenas Prácticas Implementadas	18
7.2.	Advertencias de Seguridad	19
8.	Resumen de Librerías por Componente	19
8.1.	Python Backend	19
8.2.	Arduino ESP32 Botón	19
8.3.	Arduino ESP32-CAM	19
8.4.	Frontend JavaScript	19
9.	Conclusión	19

1. Resumen del Proyecto

EyeDooR es un sistema inteligente de timbre con cámara que integra:

- **Backend:** Servidor web Flask que gestiona autenticación, captura de fotos y base de datos
- **Frontend:** Interfaz web responsive con streaming de video en vivo
- **Hardware:** ESP32-CAM para video + ESP32 con botón de timbre conectados a Firebase

1.1. Flujo General del Sistema

El sistema funciona con el siguiente flujo:

1. Usuario presiona timbre físico
2. ESP32 Botón detecta la pulsación
3. Se envía evento a Firebase Realtime Database
4. Frontend detecta el cambio en Firebase
5. Backend Flask captura foto desde ESP32-CAM
6. ESP32-CAM toma la foto
7. La imagen se guarda en disco local y Firebase

Adicionalmente, el usuario puede ver el stream de video en vivo desde el navegador y presionar el botón web para tomar fotos manualmente.

2. Backend (Python/Flask)

2.1. Archivo Principal: app.py

El backend es un servidor web construido con Flask que proporciona:

- Sistema de autenticación de usuarios
- Integración con Firebase Realtime Database
- Captura de fotos desde ESP32-CAM
- API REST para el frontend

2.2. Paquetes y Librerías del Backend

2.2.1. Flask (flask)

```
1 from flask import Flask, render_template, request, redirect,
2   url_for, flash, session, jsonify
```

Propósito: Framework web principal que proporciona:

- **Flask:** Clase principal para crear la aplicación web
- **render_template:** Renderiza plantillas HTML con datos dinámicos
- **request:** Accede a datos de solicitudes HTTP (formularios, query params)
- **redirect:** Redirige al usuario a otra ruta
- **url_for:** Genera URLs dinámicas para rutas
- **flash:** Muestra mensajes temporales al usuario (éxito, error)
- **session:** Maneja sesiones de usuario (mantiene login activo)
- **jsonify:** Convierte datos Python a formato JSON para APIs

Uso en EyeDooR: Base del servidor que maneja todas las rutas (/login, /registro, /bienvenido, /capturar).

2.2.2. Werkzeug (werkzeug.security)

```
1 from werkzeug.security import generate_password_hash,
2   check_password_hash
```

Propósito: Librería de seguridad para manejar contraseñas:

- **generate_password_hash:** Encripta contraseñas usando algoritmos seguros (bcrypt)
- **check_password_hash:** Verifica si una contraseña coincide con el hash almacenado

Uso en EyeDooR:

- Al registrarse: `password_hash = generate_password_hash(password)`
- Al iniciar sesión: `check_password_hash(user['password_hash'], password)`
- También hashea las respuestas de seguridad para recuperación de contraseña

Seguridad: Jamás almacena contraseñas en texto plano.

2.2.3. Firebase Admin SDK (firebase-admin)

```
1 import firebase_admin
2 from firebase_admin import credentials, db
```

Propósito: SDK oficial de Google para interactuar con Firebase desde Python:

- `credentials.Certificate`: Carga credenciales JSON para autenticación
- `db.reference`: Crea referencias a nodos de la base de datos en tiempo real
- `db.get()`: Lee datos de Firebase
- `db.set()`: Escribe datos en Firebase
- `db.push()`: Agrega nuevos datos con ID único auto-generado

Uso en EyeDooR:

```
1 # Inicializaci3n
2 cred = credentials.Certificate("eyedoor-firebase.json")
3 firebase_admin.initialize_app(cred, {
4     'databaseURL': 'https://eyedoor-49c27-default-rtdb.firebaseio.com/'
5 })
6
7 # Operaciones
8 ref = db.reference("users") # Referencia a usuarios
9 ref.push({...})            # Crear nuevo usuario
10 ref.get()                  # Leer todos los usuarios
11 db.reference("fotos_eyedoor").push({...}) # Guardar metadatos de fotos
```

Base de datos Firebase: Almacena usuarios, códigos de producto, fotos, y eventos de timbre.

2.2.4. Requests (requests)

```
1 import requests
```

Propósito: Librería HTTP para hacer solicitudes a servidores externos.

Uso en EyeDooR:

```
1 CAMERA_CAPTURE_URL = "http://10.154.150.123:81/capture"
2 response = requests.get(CAMERA_CAPTURE_URL, timeout=15)
```

- Se conecta al ESP32-CAM en el puerto 81
- Descarga la imagen JPEG capturada
- Timeout de 15 segundos para evitar bloqueos

2.2.5. OS (os)

```
1 import os
```

Propósito: Librería estándar de Python para operaciones del sistema operativo.

Uso en EyeDooR:

```
1 FOLDER = "fotos_eyedoor"
2 os.makedirs(FOLDER, exist_ok=True) # Crea carpeta si no existe
3 filepath = os.path.join(FOLDER, filename) # Une rutas de forma segura
```

2.2.6. RE (re)

```
1 import re
2 CODE_REGEX = re.compile(r'^[A-Z0-9]{5}-[A-Z0-9]{5}-[A-Z0-9]{5}$')
```

Propósito: Librería de expresiones regulares para validación de patrones.

Uso en EyeDooR: Valida formato de códigos de producto (ej: ABCDE-FGHIJ-KLMNO).

2.2.7. Datetime (datetime)

```
1 from datetime import datetime
2 filename = datetime.now().strftime("%Y-%m-%d_%H_%M_%S") + ".jpg"
```

Propósito: Manejo de fechas y horas.

Uso en EyeDooR: Genera nombres únicos para fotos (ej: 2025-12-03_00_27_15.jpg).

2.3. Rutas Principales del Backend

2.3.1. Autenticación

- GET/POST /login: Inicio de sesión con email y contraseña
- GET/POST /registro: Registro de nuevos usuarios con código de producto
- GET/POST /recuperar: Recuperación de contraseña usando pregunta de seguridad
- GET /logout: Cierra sesión y limpia session

2.3.2. Panel Principal

- GET /bienvenido: Panel principal con stream de cámara (requiere login)

2.3.3. API de Cámara

GET /capturar: Endpoint crítico que:

1. Se conecta al ESP32-CAM en `http://10.154.150.123:81/capture`
2. Descarga la imagen JPEG
3. La guarda localmente en `fotos_eyedoor/`
4. Sube metadatos a Firebase
5. Retorna JSON: `{"success": true, "file": "nombre.jpg"}`

2.3.4. Debug

- GET `/_debug_list_users`: Lista todos los usuarios registrados (solo para desarrollo)

2.4. Sistema de Validación de Códigos de Producto

```
1 VALID_PRODUCT_CODES = {  
2     "ABCDE-FGHIJ-KLMNO",  
3     "AAAAA-BBBBBB-CCCCC",  
4     "FFFFFF-GGGGGG-HHHHH",  
5     "12345-ABCDE-99999"  
6 }
```

- Cada código solo puede usarse una vez
- Se valida en el registro con `is_product_code_used()`
- Asocia cada usuario a un dispositivo físico único

3. Frontend (HTML/CSS/JavaScript)

3.1. Archivos de Plantillas (carpeta templates/)

El frontend utiliza **Jinja2** (motor de plantillas de Flask) para HTML dinámico.

3.1.1. login.html

Propósito: Página de inicio de sesión.

Características:

- Formulario con email y contraseña
- Validación del lado del cliente
- Enlaces a registro y recuperación
- Mensajes flash para feedback

3.1.2. registro.html

Propósito: Registro de nuevos usuarios.

Campos del formulario:

- Email
- Contraseña (mínimo 6 caracteres)
- Confirmación de contraseña
- Código de producto (formato: XXXXX-XXXXX-XXXXX)
- Pregunta de seguridad personalizada

- Respuesta de seguridad

Validaciones:

- Formato de código de producto
- Código no usado previamente
- Email único
- Contraseñas coincidentes

3.1.3. recuperar.html

Propósito: Recuperación de contraseña olvidada.

Flujo:

1. Usuario ingresa su email
2. Sistema muestra su pregunta de seguridad
3. Usuario responde
4. Si es correcta, puede establecer nueva contraseña

3.1.4. bienvenido.html (Página Principal)

Esta es la página más compleja e importante del sistema.

Estructura HTML

```
1 <div class="panel">
2   <!-- Lado izquierdo: Stream de video -->
3   <div class="camera-box">
4     <iframe src="http://10.154.150.123"
5       width="100%" height="100%">
6     </iframe>
7   </div>
8
9   <!-- Lado derecho: Controles -->
10  <div class="right-column">
11    <div id="timbre-notificacion"></div>
12    <div id="mensaje"></div>
13    <button id="btnFoto">          Fotografiar</button>
14    <div class="info">Email y código de producto</div>
15    <a href="/logout">Cerrar sesión</a>
16  </div>
17 </div>
```

Librerías JavaScript del Frontend Firebase JavaScript SDK (v8.10.0)

```
1 <script src="https://www.gstatic.com/firebasejs/8.10.0/firebase-app.js">
2 </script>
3 <script src="https://www.gstatic.com/firebasejs/8.10.0/firebase-database
4   .js">
5 </script>
```

Propósito: SDK de Firebase para navegadores web.

Módulos usados:

- `firebase-app.js`: Núcleo de Firebase
- `firebase-database.js`: Realtime Database (escucha cambios en tiempo real)

Configuración:

```
1 const firebaseConfig = {
2   apiKey: "AIzaSyDQ9Vq0xYg_e82XK0KajuLtlt8YRanxVdw",
3   authDomain: "eyedoor-49c27.firebaseio.com",
4   databaseURL: "https://eyedoor-49c27-default-rtdb.firebaseio.com/",
5   projectId: "eyedoor-49c27",
6   storageBucket: "eyedoor-49c27.firebaseio.com",
7   messagingSenderId: "176516386747",
8   appId: "1:176516386747:web:2f6cc69249f845cd28be9e"
9 };
10 firebase.initializeApp(firebaseConfig);
11 const db = firebase.database();
```

Uso en EyeDooR:

- Escucha cambios en el nodo `/timbres` de Firebase
- Detecta cuando el ESP32 del botón envía un evento
- Dispara automáticamente la captura de foto

Lógica JavaScript Principal 1. Detección de Eventos de Timbre

```
1 db.ref("timbres").on("value", snapshot => {
2   const data = snapshot.val();
3
4   // Ignorar primer evento (carga inicial)
5   if (firstLoad) {
6     firstLoad = false;
7     return;
8   }
9
10  // Nuevo evento detectado - disparar captura
11  tomarFotoAutomaticamente('timbre');
12 });
```

Concepto Clave: Listener en tiempo real que reacciona instantáneamente cuando el ESP32 escribe en Firebase.

2. Función de Captura de Foto

```

1 function tomarFotoAutomaticamente(origen) {
2   // Evitar clicks mltiples
3   if (btnFoto.classList.contains("active")) return;
4
5   btnFoto.classList.add("active");
6
7   // Mensaje seg n origen
8   mensaje.textContent = origen === 'timbre'
9     ? "          Timbre presionado.          Tomando foto..."
10    : "          Tomando foto...";
11
12   // Llamar al backend Flask
13   fetch("/capturar")
14     .then(r => r.ok
15       ? mensaje.textContent = (origen === 'timbre'
16         ? "          Timbre presionado, foto tomada."
17         : "          Foto tomada.")
18       : r.json().then(data =>
19         mensaje.textContent = '          Error: ${data.error}'))
20     )
21     .catch(() => mensaje.textContent =
22       "          No se pudo contactar el servidor Flask.")
23     .finally(() => setTimeout(() => {
24       btnFoto.classList.remove("active");
25       setTimeout(() => mensaje.textContent = '', 5000);
26     }, 1000));
27 }

```

Fetch API: Hace solicitud HTTP GET a /capturar del backend Flask.

3. Botón Manual de Fotografía

```

1 btnFoto.addEventListener("click", () => {
2   tomarFotoAutomaticamente('manual');
3 });

```

Dual Trigger: La foto puede dispararse por:

1. **Timbre físico** → Firebase listener → Automático
2. **Botón web** → Click manual → Fetch directo

Stream de Video en Vivo

```

1 <iframe src="http://10.154.150.123" width="100%" height="100%">
2 </iframe>

```

Conexión directa al ESP32-CAM puerto 80 que transmite MJPEG stream.

Prevención de Navegación con Caché

```

1 // Evita que el usuario vuelva atras sin hacer logout
2 history.pushState(null, null, location.href);
3 window.onpopstate = function () {
4   history.pushState(null, null, location.href);
5 };
6
7 // Recarga la pagina si viene del cache del navegador

```

```

8 window.addEventListener("pageshow", function (event) {
9     if (event.persisted) {
10         window.location.reload();
11     }
12 });

```

Propósito de seguridad: Fuerza a los usuarios a usar el botón “Cerrar sesión”.

3.2. Estilo CSS

Diseño: Dark theme moderno con:

- Fondo negro (#111)
- Panel central con glassmorphism
- Botones con estados (gris inactivo → rojo activo)
- Tipografía: Poppins (Google Fonts implícita)

4. Hardware (Arduino/ESP32)

El sistema utiliza **dos ESP32** separados:

1. **ESP32 con botón** (boton.ino) - Detecta pulsaciones físicas
2. **ESP32-CAM** (camara.ino) - Streaming y captura de fotos

4.1. Hardware 1: ESP32 con Botón de Timbre

4.1.1. Archivo: arduino/boton/boton.ino

4.1.2. Librerías de Arduino del Botón

WiFi.h

```

1 #include <WiFi.h>

```

Propósito: Conectividad WiFi para ESP32.

Uso:

```

1 WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
2 while (WiFi.status() != WL_CONNECTED) { delay(300); }

```

time.h

```

1 #include <time.h>

```

Propósito: Librería estándar de C para manejo de tiempo.

Funciones usadas:

- `struct tm`: Estructura para fecha/hora
- `getLocalTime()`: Obtiene hora del sistema
- `strftime()`: Formatea fecha a string
- `configTime()`: Configura servidor NTP

FirestoreESP32.h (por Mobizt)

```

1 #include "FirestoreESP32.h"
2 #include <addons/RTDBHelper.h>
3 #include <addons/TokenHelper.h>

```

Propósito: Cliente de Firestore para ESP32 (versión Mobizt).

Componentes:

- FirestoreESP32.h: Core de la librería
- RTDBHelper.h: Helpers para Realtime Database
- TokenHelper.h: Manejo de tokens de autenticación

Objetos globales:

```

1 FirestoreData firestoreData; // Buffer de datos
2 FirebaseAuth auth; // Autenticación
3 FirestoreConfig config; // Configuración

```

Autenticación:

```

1 config.api_key = API_KEY;
2 auth.user.email = USER_EMAIL;
3 auth.user.password = USER_PASSWORD;
4 FirebaseAuth.begin(&config, &auth);

```

Escritura en Firestore:

```

1 FirestoreJson json;
2 json.set("evento", "Timbre presionado");
3 json.set("timestamp", timestamp);
4 FirebaseAuth.setJSON(firestoreData, "/timbres", json);

```

4.1.3. Configuración del Hardware

```

1 #define WIFI_SSID "Antonio"
2 #define WIFI_PASSWORD "12345678"
3 const int PUSH_BUTTON_PIN = 4; // GPIO 4 con pull-up interno

```

Debounce: 2 segundos entre pulsaciones para evitar rebotes.

4.1.4. Sincronización NTP

```

1 const char *ntpServer = "pool.ntp.org";
2 const long gmtOffset_sec = -18000; // GMT-5 (Colombia/Per /Ecuador)
3 const int daylightOffset_sec = 0;

```

Función crítica: syncTimeNTP()

- 60 intentos de 500ms = 30 segundos máximo
- Servidor primario: pool.ntp.org
- Servidor de respaldo: time.google.com

- Verifica conexión WiFi durante sincronización

Timestamp generado:

```

1 String getFormattedDateTime() {
2     struct tm timeinfo;
3     if (!getLocalTime(&timeinfo)) return "Error: Sin hora";
4
5     char timeString[50];
6     strftime(timeString, sizeof(timeString),
7             "%Y-%m-%d %H:%M:%S", &timeinfo);
8     return String(timeString);
9 }
```

Formato: 2025-12-03 00:27:15

4.1.5. Loop Principal del Botón

```

1 void loop() {
2     int buttonState = digitalRead(PUSH_BUTTON_PIN);
3
4     if (buttonState == LOW) { // Botón presionado (pull-up)
5         if (millis() - lastPressTime > debounceDelay) {
6             lastPressTime = millis();
7
8             String timestamp = getFormattedDateTime();
9
10            FirebaseJson json;
11            json.set("evento", "Timbre presionado");
12            json.set("timestamp", timestamp);
13
14            if (Firebase.setJSON(firebaseData, "/timbres", json)) {
15                Serial.println(" Datos enviados a Firebase exitosamente
16                               !");
17            }
18        }
19    }
```

Flujo:

1. Lee estado del botón GPIO 4
2. Si está presionado (LOW debido a pull-up)
3. Verifica debounce (2 segundos desde última pulsación)
4. Obtiene timestamp actual con NTP
5. Crea JSON con evento y timestamp
6. Envía a Firebase `/timbres`
7. Frontend detecta el cambio y dispara captura

4.2. Hardware 2: ESP32-CAM

4.2.1. Archivo: arduino/camara/camara.ino

4.2.2. Librerías de Arduino de la Cámara

esp_camera.h

```
1 #include "esp_camera.h"
```

Propósito: Driver oficial de Espressif para cámara OV2640.

Funciones principales:

- esp_camera_init(): Inicializa la cámara con configuración
- esp_camera_fb_get(): Captura un frame buffer
- esp_camera_fb_return(): Libera memoria del frame

Configuración de pines AI-Thinker:

```
1 #define PWDN_GPIO_NUM    32
2 #define RESET_GPIO_NUM  -1
3 #define XCLK_GPIO_NUM    0
4 // ... 19 pines en total
```

WiFi.h

```
1 #include <WiFi.h>
```

Misma función que en el botón (conectividad WiFi).

esp_http_server.h

```
1 #include "esp_http_server.h"
```

Propósito: Servidor HTTP nativo de ESP-IDF.

Tipos de datos:

- httpd_req_t*: Request HTTP
- httpd_handle_t: Handle del servidor
- httpd_config_t: Configuración del servidor

Funciones:

- httpd_start(): Inicia servidor en un puerto
- httpd_register_uri_handler(): Registra endpoints
- httpd_resp_set_type(): Establece Content-Type
- httpd_resp_send(): Envía respuesta HTTP

freertos/semphr.h

```

1 #include "freertos/semphr.h"
2 SemaphoreHandle_t camera_mutex;

```

Propósito: Semáforos de FreeRTOS para sincronización.

Problema resuelto: La cámara no puede ser usada simultáneamente por:

- Stream de video (puerto 80)
- Captura de foto (puerto 81)

Solución con MUTEX:

```

1 xSemaphoreTake(camera_mutex, portMAX_DELAY); // Bloquear
2 fb = esp_camera_fb_get();                    // Usar cámara
3 xSemaphoreGive(camera_mutex);                // Liberar

```

4.2.3. Configuración de la Cámara

```

1 camera_config_t createCameraConfig() {
2     camera_config_t config;
3     config.pixel_format = PIXFORMAT_JPEG;
4     config.frame_size   = FRAMESIZE_QVGA; // 320x240
5     config.jpeg_quality = 10;              // 0-63, menor = mejor
6     config.fb_count     = 2;              // 2 buffers para stream
7                                     fluido
8     config.xclk_freq_hz = 200000000;      // 20 MHz
9     // ... configuración de pines
10    return config;
11 }

```

- **FRAMESIZE_QVGA:** Resolución 320x240px (rápida y eficiente)
- **jpeg_quality = 10:** Alta calidad (rango 0-63)
- **fb_count = 2:** Double buffering para stream sin cortes

4.2.4. Arquitectura de Dual-Server**Puerto 80: Stream de Video**

```

1 static esp_err_t stream_handler(httpd_req_t *req) {
2     httpd_resp_set_type(req,
3         "multipart/x-mixed-replace; boundary=frame");
4
5     while (true) {
6         xSemaphoreTake(camera_mutex, portMAX_DELAY);
7         fb = esp_camera_fb_get();
8         xSemaphoreGive(camera_mutex);
9
10        httpd_resp_send_chunk(req,
11            "--frame\r\nContent-Type: image/jpeg...");
12        httpd_resp_send_chunk(req,
13            (const char *)fb->buf, fb->len);
14    }

```

```

15     esp_camera_fb_return(fb);
16     vTaskDelay(pdMS_TO_TICKS(50)); // 50ms delay = ~20 FPS
17 }
18 }

```

MJPEG Stream: Secuencia infinita de JPEGs con boundary `--frame`.

`vTaskDelay(50ms)`:

- Permite que otras tareas se ejecuten
- Evita bloqueo del CPU
- ~20 FPS es suficiente para video fluido

Puerto 81: Captura de Foto

```

1 static esp_err_t capture_handler(httpd_req_t *req) {
2     xSemaphoreTake(camera_mutex, portMAX_DELAY);
3     fb = esp_camera_fb_get();
4     xSemaphoreGive(camera_mutex);
5
6     httpd_resp_set_type(req, "image/jpeg");
7     httpd_resp_set_hdr(req, "Content-Disposition",
8         "inline; filename=capture.jpg");
9     httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
10
11     httpd_resp_send(req, (const char *)fb->buf, fb->len);
12     esp_camera_fb_return(fb);
13
14     return ESP_OK;
15 }

```

Headers importantes:

- **Content-Type:** `image/jpeg`: Indica que es una imagen
- **Access-Control-Allow-Origin:** `*`: Permite peticiones desde Flask
- **Content-Disposition:** `inline`: Muestra en navegador (no descarga)

4.2.5. Inicialización de Servidores

```

1 void startCameraServer() {
2     httpd_config_t config = HTTPD_DEFAULT_CONFIG();
3
4     // Servidor 1: Stream (puerto 80)
5     config.server_port = 80;
6     httpd_start(&stream_httpd, &config);
7     httpd_register_uri_handler(stream_httpd, &uri_stream);
8
9     // Servidor 2: Captura (puerto 81)
10    config.server_port = 81;
11    config.ctrl_port = 32769; // Puerto de control diferente
12    httpd_start(&capture_httpd, &config);
13    httpd_register_uri_handler(capture_httpd, &uri_capture);
14 }

```

`ctrl_port = 32769`: Evita conflicto entre ambos servidores HTTP.

4.2.6. Endpoints de la ESP32-CAM

Endpoint	Puerto	Función
<code>http://10.154.150.123/</code>	80	Stream MJPEG continuo
<code>http://10.154.150.123:81/capture</code>	81	Captura una sola foto

5. Flujo Completo de Captura de Foto

5.1. Escenario 1: Timbre Físico Presionado

1. Visitante presiona timbre físico
2. ESP32 Botón solicita timestamp al servidor NTP
3. NTP responde con hora actual (ej: 2025-12-03 00:27:15)
4. ESP32 Botón envía JSON a Firebase: `{evento, timestamp}`
5. Firebase dispara evento “value” al Frontend
6. Frontend detecta nuevo evento
7. Frontend hace GET a `/capturar` del Backend
8. Backend hace GET a `http://10.154.150.123:81/capture`
9. ESP32-CAM llama a `esp_camera_fb_get()`
10. ESP32-CAM retorna imagen JPEG al Backend
11. Backend guarda imagen en disco: `2025-12-03_00_27_15.jpg`
12. Backend sube metadatos a Firebase: `fotos_eyedoor`
13. Backend retorna JSON al Frontend: `{success: true, file: "..."}`
14. Frontend muestra mensaje: “ Timbre presionado, foto tomada.”

5.2. Escenario 2: Botón Web Presionado

1. Usuario web hace click en “ Fotografiar”
2. Frontend hace GET a `/capturar` del Backend
3. Backend hace GET a `http://10.154.150.123:81/capture`
4. ESP32-CAM llama a `esp_camera_fb_get()`
5. ESP32-CAM retorna imagen JPEG al Backend
6. Backend guarda imagen en disco: `2025-12-03_00_27_20.jpg`

7. Backend sube metadatos a Firebase: `fotos_eyedoor`
8. Backend retorna JSON al Frontend: `{success: true, file: "..."}`
9. Frontend muestra mensaje: “ Foto tomada.”

6. Estructura de Datos en Firebase

6.1. Nodo /users

```
1 {  
2   "-NxYz123abc": {  
3     "email": "usuario@example.com",  
4     "password_hash": "$2b$12$...",  
5     "product_code": "ABCDE-FGHIJ-KLMNO",  
6     "security_question": " Nombre de tu mascota?",  
7     "security_answer_hash": "$2b$12$..."  
8   }  
9 }
```

6.2. Nodo /timbres

```
1 {  
2   "evento": "Timbre presionado",  
3   "timestamp": "2025-12-03 00:27:15"  
4 }
```

6.3. Nodo /fotos_eyedoor

```
1 {  
2   "-NxYz456def": {  
3     "file": "2025-12-03_00_27_15.jpg",  
4     "timestamp": "2025-12-03T05:27:15.123456",  
5     "uploader": "usuario@example.com"  
6   }  
7 }
```

7. Consideraciones de Seguridad

7.1. Buenas Prácticas Implementadas

1. Contraseñas hasheadas: Werkzeug bcrypt
2. Sesiones seguras: Flask session con `secret_key`
3. Validación de códigos: Regex + lista blanca
4. CORS configurado: Access-Control-Allow-Origin en ESP32-CAM
5. Timeouts: 15 segundos en requests a ESP32

7.2. Advertencias de Seguridad

ADVERTENCIA: Las credenciales de Firebase están hardcodeadas en el código. En producción, usar variables de entorno.

PRECAUCIÓN: La IP 10.154.150.123 es local. No funcionará fuera de la red WiFi “Antonio”.

8. Resumen de Librerías por Componente

8.1. Python Backend

Librería	Versión	Propósito
Flask	~3.0	Framework web
firebase-admin	~6.0	SDK Firebase para Python
Werkzeug	~3.0	Seguridad (incluido con Flask)
requests	~2.31	Cliente HTTP

8.2. Arduino ESP32 Botón

Librería	Autor	Propósito
WiFi.h	Espressif	Conectividad WiFi
time.h	Estándar C	Manejo de tiempo/NTP
FirebaseESP32	Mobizt	Cliente Firebase

8.3. Arduino ESP32-CAM

Librería	Autor	Propósito
esp_camera.h	Espressif	Driver de cámara OV2640
WiFi.h	Espressif	Conectividad WiFi
esp_http_server.h	Espressif	Servidor web HTTP
freertos/semphr.h	FreeRTOS	Semáforos para sincronización

8.4. Frontend JavaScript

9. Conclusión

EyeDooR es un sistema distribuido que combina:

- **Backend robusto** con autenticación y almacenamiento

Librería	Versión	Propósito
Firebase JS SDK	8.10.0	Cliente Firebase web

- **Frontend reactivo** con actualizaciones en tiempo real
- **Hardware eficiente** con dual-server y sincronización NTP
- **Base de datos en la nube** para persistencia

El flujo principal (Timbre → Firebase → Web → Flask → ESP32-CAM → Foto) demuestra una arquitectura bien diseñada con separación de responsabilidades y comunicación eficiente entre componentes.